

Bill Scraper: Architecture

This scraping example scrapes all of the bills in the 2025-2026 session of the Pennsylvania state Senate, using this page as the contents page:

<https://www.palegis.us/legislation/bills/bill-index?display=index&sessYr=2025&sessInd=0&billBody=S&filter=bills>

This document contains just two cells:

extract_data_points() function: this is a function that handles extracting the contents of each page as the scraper moves from page to page. All this code is hyper-specific to the structure of the bill pages on the Pennsylvania bill website, as is the case with all scraping. The main aspect of this that matters to you specifically, is how thorough it is in extracting as many of the key data points from each page that we may want to use. It attempts to get almost every field, and every aspect of every field, without breaking. That takes a balance of specificity and generality in using the HTML tags and strings that structure the pages that contain bills.

Scraping Architecture: this is the one to very thoroughly study because you want to follow this architecture. It sets up the following output documents that go into a python-generated folder I have name *data*:

-scraped data output: in json--called *pa_senate_bills.json* -change log: logs the different changes (additions, deletions, modifications) between the current scrape in the last one. -error log: logs the errors from page to page if they occur (in the **except** block in the try/except structure)

Your goal for this weekend is just to try to find the source and figure out how you're going to scrape it--and to begin writing that code. But ultimately you want to follow this essential architecture.

```
In [ ]: ###FUNCTION FOR EXTRACTING DATA POINTS FROM EACH BILL PAGE

import re
from bs4 import BeautifulSoup

def extract_data_points(bill_page,url,hash_id,bill_id):
    fields = {}
    fields["content_hash"]=hash_id
    fields["url"]=url
    fields["bill_id"]=link.text
    fields["bill_name"]=bill_page.find(class_='h1 header-title').text.
```

```

fields["short_des"]=bill_page.find(id="shortTitle-wrapper").text.s
if bill_page.find(class_="col-12 ps-2"):
    fields["memo_name"]=bill_page.find(class_="col-12 ps-2").text.
    fields["memo_link"]=bill_page.find(class_="col-12 ps-2").a['hr
prime_container = bill_page.find(string="Prime Sponsor").parent.fi
fields["prime_sponsor"]={
    "name": prime_container.strong.text.strip(),
    "party": prime_container.span.text.strip(),
    "district": prime_container.span.next_sibling.strip(),
}
if bill_page.find(string="Co-Sponsors"):
    co_sponsors=bill_page.find(string="Co-Sponsors").parent.find_n
    all_cosp=co_sponsors.find_all(class_=re.compile("^col"))
    fields["cosponsors"]=[]
    for cosp in all_cosp:
        each_cosp= {
            "name": cosp.strong.text.strip(),
            "party": cosp.span.text.strip(),
            "district": cosp.span.next_sibling.strip()
        }
        fields["cosponsors"].append(each_cosp)
last_action=bill_page.find(string=re.compile("Last Action")).paren
fields["last_action"]=" ".join(last_action.split())
timeline=bill_page.find(class_=re.compile("timeline"))
all_items = timeline.find_all('span')
time_dict = {}
for item in all_items:
    item_html = BeautifulSoup(item['title'], "html.parser")
    time_elements = item_html.find_all(string=True)
    time_key = time_elements.pop(0)
    time_dict[time_key] = time_elements
fields["timeline"] =time_dict
history_table=bill_page.find(string=re.compile("View Full History")
actions_list = []
for row in history_table.find_all('tr'):
    actions_dict = {}
    actions_dict['action']=row.text.strip()
    if row.td.a:
        actions_dict['doc']=row.td.a['aria-label']
        actions_dict['doc_link']=row.td.a['href']
    actions_list.append(actions_dict)
fields["actions"]=actions_list
main_votes=bill_page.find(id="VotesSection").find_next_sibling('di
fields["votes"] = []
for vote in main_votes:
    vt={}
    vt['date']=vote.find(class_=re.compile("^h5")).text
    vt['type']=vote.find(class_=re.compile("^h6")).text
    vt['bill']=vote.find(class_=re.compile("^mb-3")).text
    vt['link']=vote.a['href']
    vote_tally=vote.find_all(class_=re.compile("^fa-solid"))
    for item in vote_tally:

```

```

        vt[item.find_next_siblings('span')[1].text]=item.find_next
        fields["votes"].append(vt)
    if bill_page.find(id="billVotes"):
        add_votes=bill_page.find(id="billVotes").table.find_all('tr')
        for vote in add_votes:
            vt={}
            date_type=vote.find(class_=re.compile("^h6")).text
            vt['date']=date_type.split("-")[0].strip()
            vt['type']=date_type.split("-")[1].strip()
            vt['bill']=vote.a.text
            vt['link']=vote.a['href']
            vt['YES Votes']=vote.find(class_=re.compile("fa-square-che
            vt['NO Votes']=vote.find(class_=re.compile("fa-square-xmar
            fields["votes"].append(vt)
    return fields

```

```

In [ ]: #####SCRAPING LOGIC#####

#####IMPORTS#####

import os
import json
import hashlib
import datetime
import traceback
import requests
import time
import re
from bs4 import BeautifulSoup

#####STEP ONE#####
#check for data history

DATA_FILE = "data/pa_senate_bills.json" #FINAL OUTPUT OF SCRAPED DATA
TODAY_STR = datetime.date.today().isoformat()

# CHECKING / BUILDING DIRECTORIES FOR DATA, CHANGE LOG, AND ERROR
os.makedirs("data/changelogs", exist_ok=True)
os.makedirs("data/error_logs", exist_ok=True)
os.makedirs(os.path.dirname(DATA_FILE), exist_ok=True)

if os.path.exists(DATA_FILE):
    print("Found existing dataset. Loading history...")
    with open(DATA_FILE, 'r') as f:
        yesterdays_list = json.load(f)
        # Map by URL for instant lookup
        old_data_map = {item['url']: item for item in yesterdays_list}
else:
    print("No existing dataset found. Initializing a baseline run...")
    old_data_map = {} # Empty map forces EVERYTHING to be treated as a

```

#####STEP TWO#####

#Set up Change Log and Error Log

```
changelog = {
    "date": TODAY_STR,
    "additions": [],
    "deletions": [],
    "modifications": []
}
error_log = {
    "date": TODAY_STR,
    "errors": []
}
```

#####STEP THREE#####

#scrape contents page get the current contents

```
todays_bills = []
```

```
head={'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWe
```

```
url = "https://www.palegis.us/legislation/bills/bill-index?display=ind
raw_html = requests.get(url,headers=head).content
contents_doc = BeautifulSoup(raw_html, "html.parser")
bill_container = contents_doc.h3.parent.parent
all_bill_links = bill_container.find_all('a')
all_bill_urls = ["https://www.palegis.us/"+item['href'] for item in al
```

#####CHECK FOR DELETED ITEMS

```
for url, old_item in old_data_map.items():
    if url not in all_bill_urls:
        changelog["deletions"].append({
            "bill_id": old_item["bill_id"],
            "url": url,
            "title": old_item.get("title")
        })
```

#####STEP FOUR#####

#checking/scraping the bill pages

```
for link in all_bill_links[:5]:#EXAMPLE CURRENTLY JUST SCRAPE THE FIRS
    try:
        time.sleep(1)
        url = "https://www.palegis.us/"+link['href']
        raw_html = requests.get(url,headers=head).content
        bill_page = BeautifulSoup(raw_html, "html.parser")
```

```

    ###setting up hash for change detection
    history_table=bill_page.find(string=re.compile("View Full Hist
    last_action=bill_page.find(string=re.compile("Last Action")).p
    hash_string =" ".join(last_action.split())+" ".join(history_ta
    print(hash_string.lower())
    hash_id=hashlib.md5(hash_string.lower().encode('utf-8')).hexdi
    #IF THIS IS A NEW LINK--JUST GET EVERYTHING
    if url not in old_data_map:
        ###SEND THE CONTENT AND KEY DATA POINTS TO THE FUNCTION
        print("new entry")
        bill_dict=extract_data_points(bill_page,url,hash_id,link.t
        todays_bills.append(bill_dict)
        changelog["additions"].append({"bill_id": link.text, "url"
    else:
        yesterdays_item = old_data_map[url]
        yesterdays_hash = yesterdays_item['content_hash']
        #CHECK HASHES FOR CHANGE
        if yesterdays_hash == hash_id: #NO CHANGE
            print("they match!")
            todays_bills.append(yesterdays_item)
        else:
            print("no match") #THERE ARE CHANGES
            bill_dict=extract_data_points(bill_page,url,hash_id,li
            todays_bills.append(bill_dict)
            meaningful_changes = {}
            for key, value in bill_dict.items():
                if yesterdays_item.get(key) != value:
                    meaningful_changes[key] = {"from": yesterdays_
            if meaningful_changes:
                changelog["modifications"].append({"bill_id": link
except Exception as e:
    print(f"✗ Error scraping {url}: {str(e)}")
    ## AN ERROR HAPPENED ON ONE OF THE PAGES
    ## CHECK IF YOU HAVE YESTERDAY'S DICT FROM THAT PAGE AND SAVE
    if yesterdays_item:
        todays_bills.append(yesterdays_item)
    #UPDATE ERROR LOG
    error_log["errors"].append({
        "bill_id": link.text,
        "url": "https://www.palegis.us/"+link['href'],
        "error_type": type(e).__name__,
        "message": str(e),
        "traceback": traceback.format_exc().splitlines()[-3:] # Ke
    })

```

#####STEP FIVE#####

#write files

with open(DATA_FILE, 'w') as f:

```

    # Sorting by 'id' so the order remains consistant from scrape to s
    sorted_data = sorted(todays_bills, key=lambda x: x['bill_id'])

```

```
        json.dump(sorted_data, f, indent=2)
    if changelog["additions"] or changelog["deletions"] or changelog["modi
        with open(f"data/changelogs/{TODAY_STR}.json", 'w') as f:
            json.dump(changelog, f, indent=2)

# 3. Write Daily Error Log (Only write if errors occurred)
if error_log["errors"]:
    with open(f"data/error_logs/{TODAY_STR}.json", 'w') as f:
        json.dump(error_log, f, indent=2)

print("Scrape done!")
```